MOBILE APP DEVLOPEMENT

EDWARD MTACHI

402412584 Report

Contents

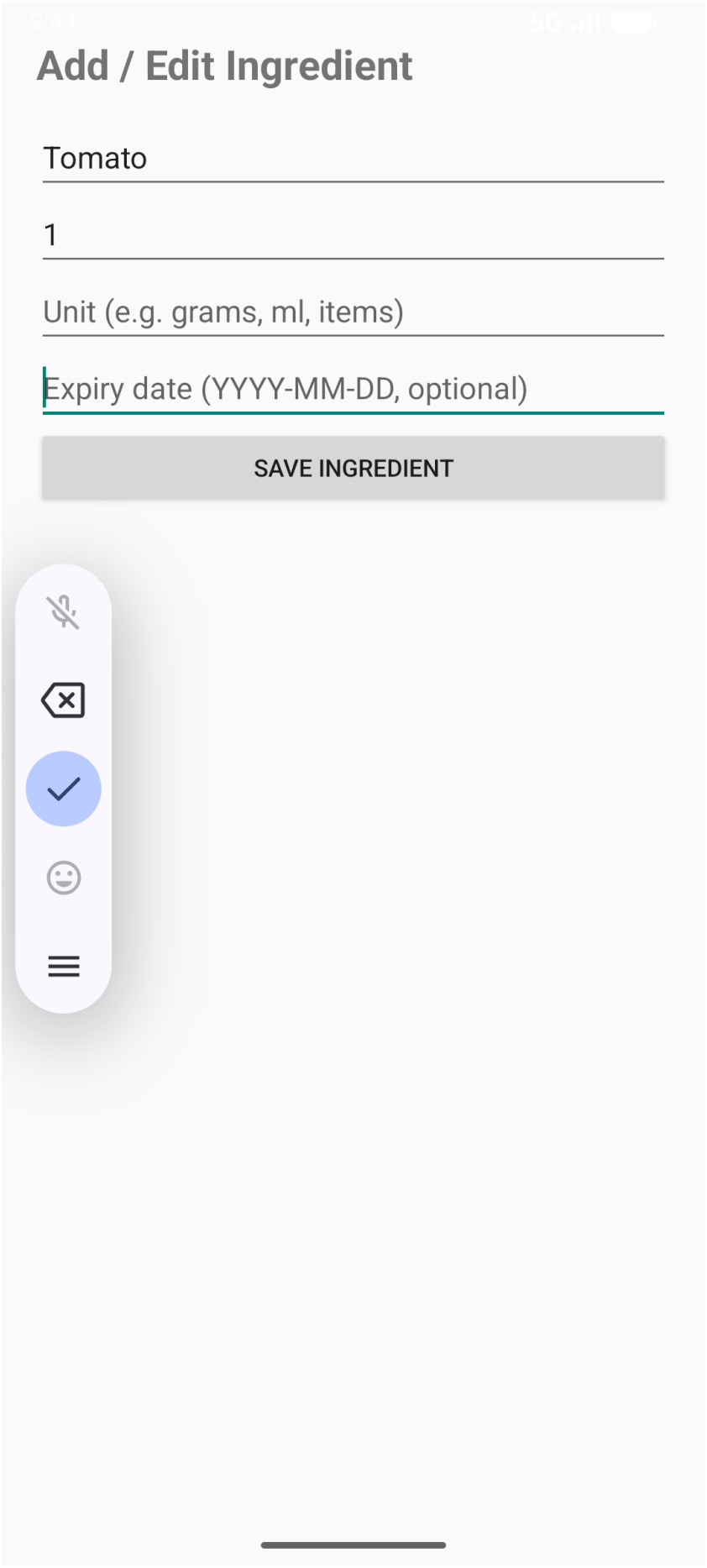
Introduction 2
System Design 2
Database Design 7
Key code Snippets 7
Challenges and Solutions 9
Conclusion and Reflection 10
References 10

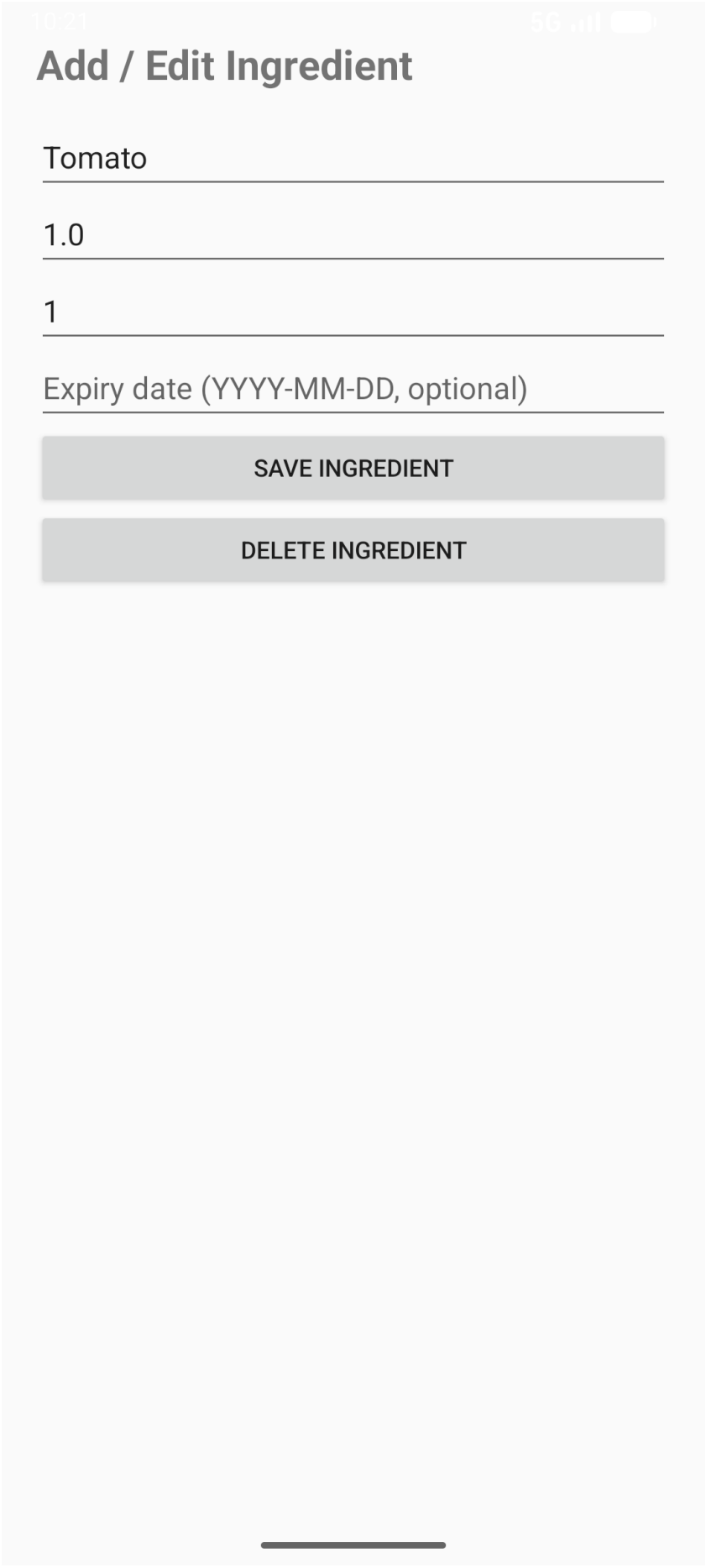
Introduction

Many people struggle with food waste whereby ingredients go bad, or the ingredients go bad after preparing one meal or people may have these ingredients but don't know what to prepare with these ingredients. But lucky for them The Smart Pantry Manger solves these problems by means of allowing users to store their ingredient list on the system so that they know what meals they can prepare with said ingredients. The target market for this application is for individuals who want to organise their pantry to make cooking and baking much easier.

System Design

This application consists of five main screens which involve the Pantry List, Add/Edit Ingredients, Suggested Recipes, Recipe Details and Settings whereby the Pantry List screen displays the ingredients saved by the user.


9:44
5G
Add / Edit Ingredient
Tomato
1
Unit (e.g. grams, ml, items)
Expiry date (YYYY-MM-DD, optional)
SAVE INGREDIENT


10:21
5G
Add / Edit Ingredient
Tomato
1.0
1
Expiry date (YYYY-MM-DD, optional)
SAVE INGREDIENT
DELETE INGREDIENT

The Add/Edit ingredients screen allows users to add or edit their ingredients.

The Suggested Recipes screen displays the recipes they can make with their existing ingredients. The Recipe Details screen provides the selected recipes ingredients and preparation instructions.

SMART PANTRY MANAGER

Storage: Local SQLite database

Pantry management: Add, edit and delete ingredients.

Recipe suggestions: A recipe is suggested only when all its required ingredients and quantities are available.

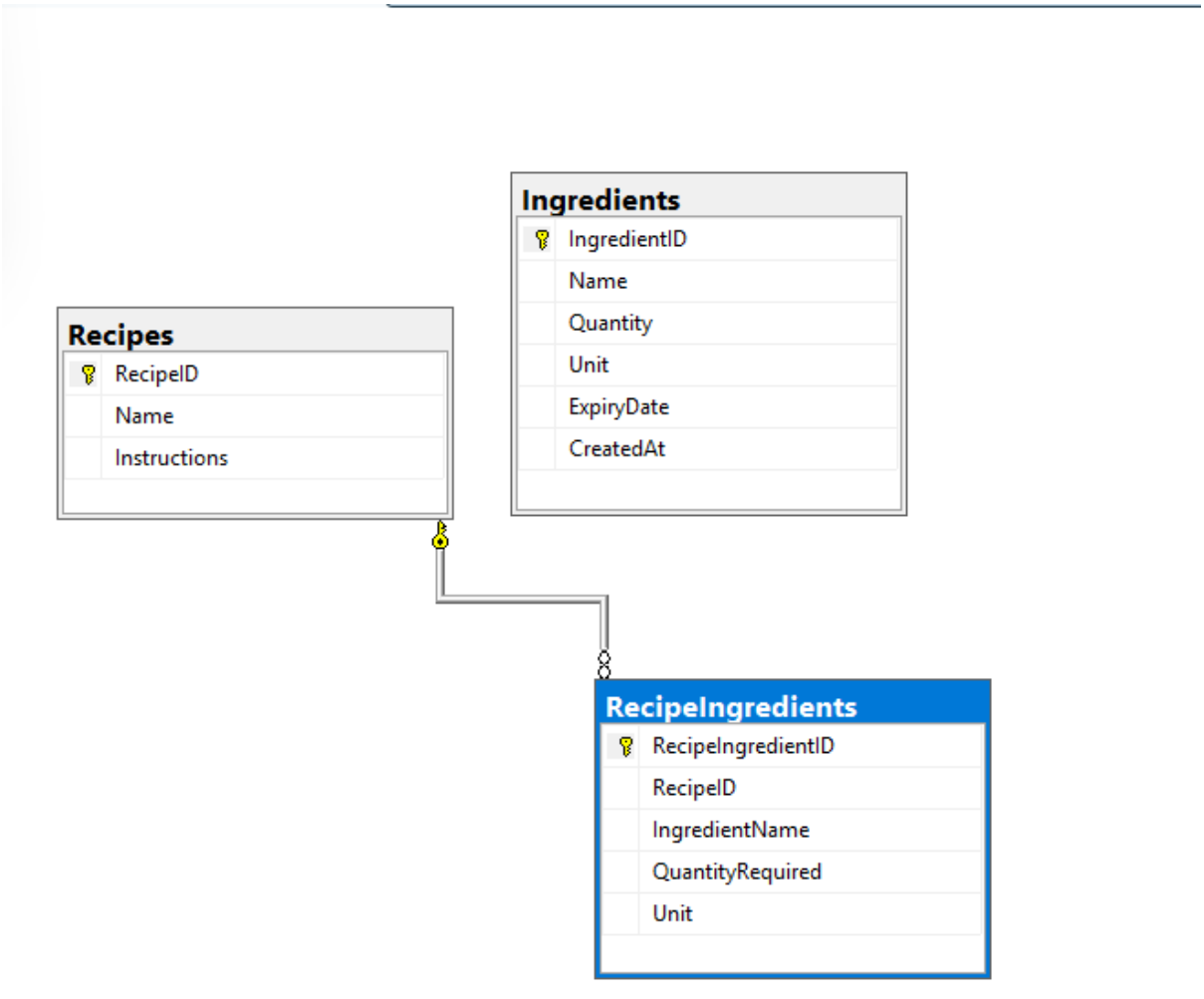
Expiry dates: Enter dates in YYYY-MM-DD format.

Recipe collection: 15 sample recipes are included.

Lastly the Setting screen shows the applications settings.

The components used to compile this application include the XML layouts that define the user interface, Java activities which was used to handle user interaction and screen navigation, Database Helper which was used to create and manage the SQLite database and lastly SQLite database which stores the pantry ingredients , recipe and recipe requirements.

Database Design



Key code Snippets

Saving an ingredient to SQLite:

ContentValues values = new ContentValues();

```
values.put("name", name);
values.put("quantity", quantity);
values.put("unit", unit);
values.put("expiry_date", expiry);

long result = getWritableDatabase()
        .insert("ingredients", null, values);
```

//The ContentValues stores the values to be saved in a database. The insert() method adds a new row to the ingredients table then the result is checked to make sure insert was successful.

Checking the recipe requirements:

```
if (availableQuantity < requiredQuantity) {
    allAvailable = false;
    break;
}

if (allAvailable) {
    matches.add(recipe);
}
```

//this code prevents partial matches from appearing in the Suggested Recipes screen so basically if one required ingredient is missing or its quantity is insufficient then the recipe is excluded.

Navigating between Activities:

```
Intent intent = new Intent(
        this, AddEditIngredientActivity.class);

intent.putExtra("ingredient_id", item.getId());
intent.putExtra("name", item.getName());
```

```
startActivity(intent);
```

//This Intent instance opens the Add/Edit Ingredient screen. The putExtra() method passes the selected ingredient's ID and name to the next Activity which allows for the form to load the correct record.

Refreshing the pantry list:

```
@Override
protected void onResume() {
   super.onResume();
   loadIngredients();
}
```

//The onResume() lifecycle callback reloads the pantry when the Main screen becomes active again this basically allows the list to reflect changes made in the Add/Edit Ingredient screen.

Challenges and Solutions

The first challenged that presented itself was trying to enforce strict recipe matching whereby a basic recipe search might show a meal even if most of the ingredients are present so the solution that was used is I implemented a matching algorithm that checks every required ingredient and its quantity so basically the meal is only shown if all the ingredients are present.

The next challenge was trying to save data in between sessions whereby information stored in Java variables can be lost when the application is closed so the only solution was to use SQLite to store pantry ingredients and recipe information.

The next challenge was handling different ingredient names whereby an ingredient can be entered as potato in the pantry but is recorded as potatoes in the recipe so the solution was to add normalisation methods to handle common singular and plural variations before compaing ingredients.

Conclusion and Reflection

In conclusion this application helps with a very underlying challenge and that Android applications can solve even more everyday challenges with proper understanding of the language you can do anything.

References

Android Developers (n.d.) *Activities*. Available at: https://developer.android.com/guide/components/activities (Accessed: 10 October 2026).

Android Developers (n.d.) *Intents and intent filters*. Available at: https://developer.android.com/guide/components/intents-filters (Accessed: 10 October 2026)

Android Developers (n.d.) *Save data using SQLite*. Available at: https://developer.android.com/training/data-storage/sqlite (Accessed: 19 October 2026)

Oracle (n.d.) *Java Documentation*. Available at: https://docs.oracle.com/en/java/ (Accessed: 23 October 2026).